

can also be used to set and read all the flag bits in the MCU, to read/write the on-chip EEPROM.

PonyProg also comes with a well-defined hardware circuit for SPI programming. The same circuit is used in the development board too by making use of MOSI, MISO, SCK and RESET pins of the MCU. To limit the line volt-

ages on the SPI interface lines, 5.1V zener diodes are used. DPDT switch S7 is used to switch between USART communication mode and SPI serial programming mode. A step-by-step procedure to program the MCU using PonyProg is provided in the latter part of the project design.

The microcontroller is clocked to a maximum frequency of 16 MIPS by using 16MHz quartz crystal X1. The crystal should always be used along with two capacitors of equal value (connected as shown in the circuit diagram) for a stable operation. The value chosen for these capacitors is 22 pF for a frequency of 16 MHz.

In order to test the MCU's ADC module, a variable voltage source is made available on-board with the help of preset VR1. Switch S6 should be kept opened when the ADC application is not in use.

The 24C04 forms the external EEPROM circuit of the development board. It has a storage capacity of 4 kB (512x8). The EEPROM can be read/written using the TWI module available in ATmega32. For this purpose, connect SDA (Serial Data) and SCL (Serial Clock Input) pins of the EEPROM to corresponding SDA and SCL pins of the MCU via a DPDT switch. Ad-

dress input pins A0, A1 and A2 of the 24C04 are held low to give a hardware address of 000.

To connect additional TWI devices to the development board, a 4-pin TWI male header strip (PC port) is provided along with power supply rails.

Construction

The entire development board circuit can be assembled on a single PCB. An actual-size, single-side PCB for the same is shown in Fig. 3 and its component layout in Fig. 4. Solder by first connecting all the jumpers in the circuit, followed by smaller-size passive components and then the bigger components.

Software program

Before loading various application machine codes into the microcontroller, you need to write the intended controller program in 'C' language. Compile it to obtain the hex code. Burn the generated hex code into the microcontroller. To perform all these tasks in this project, you can use free Open Source software like WinAVR, AVR Studio 4 and PonyProg. The programs should be installed in the PC before starting the AVR applications.

WinAVR. WinAVR (pronounced as 'whenever') is a suite of executable, Open Source software development tools for the ATMEL AVR series of RISC microprocessors hosted on the Windows platform. It includes the GNU GCC compiler for C and C++ programming languages. Visit <http://winavr.sourceforge.net/index.html> for more details.

AVR Studio 4. In this project, we have used AVR Studio 4 (Version 4.13) to compile the code written in 'C' language and generate the hex code. It can also be used to simulate the program running on the microcontroller.

PonyProg. PonyProg (Version 2.07C) is a serial device programmer software with a user-friendly GUI framework available for Windows 95, 98, 2000, NT, XP and Intel Linux. It supports I²C Bus, Microwire, SPI EEPROM, the Atmel AVR and Micro-



Fig. 6: Debug platform and device window



Fig. 7: I/O port setup in PonyProg



Fig. 8: Device selection dialogue box

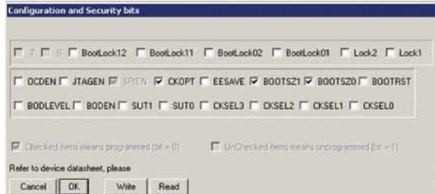


Fig. 9: Security and configuration bits setting window